departamento
de engenharia eletrónica
e telecomunicações
e de computadores

SKeyboard Reader (*Roulette Game*)
Laboratório de Informática e Computadores 2024 / 2025 verão
Autores: Margarita Sysoeva 51403 / Pilar Gonçalves 51806

O módulo *Keyboard Reader* é constituído por três blocos principais: *i*) o descodificador de teclado (*Key Decode*); *ii*) o bloco de armazenamento (designado por *Ring Buffer*); e *iii*) o bloco de entrega ao consumidor (designado por *Output Buffer*), de acordo com o diagrama representado na Figura 1. Neste caso, o módulo *Control*, implementado em software, é a entidade consumidora.

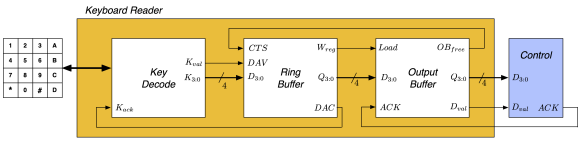
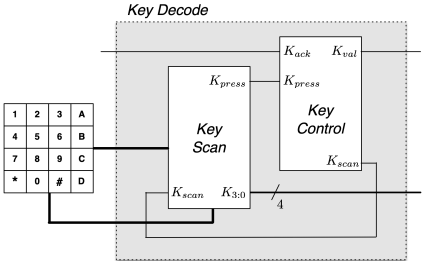
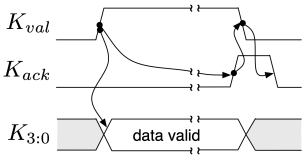


Figura 1: Diagrama de blocos do módulo *Keyboard Reader*



(a) Diagrama de blocos



(b) Diagrama temporal

Figura 2: Bloco *Key Decode*

1 Key Decode

O bloco *Key Decode* implementa um descodificador de um teclado matricial 4x4 por hardware, sendo constituído por três sub-blocos: *i*) um teclado matricial de 4x4; *ii*) o bloco *Key Scan*, responsável pelo varrimento do teclado; e *iii*) o bloco *Key Control*, que realiza o controlo do varrimento e o controlo de fluxo, conforme o diagrama de blocos representado na Figura 2a.

O controlo de fluxo de saída do bloco *Key Decode* (para o módulo *Key Buffer*) define que o sinal K_{val} é ativado quando é detetada a pressão de uma tecla, sendo também disponibilizado o código dessa tecla no barramento $K_{0:3}$. Apenas é iniciado um novo ciclo de varrimento ao teclado quando o sinal K_{ack} for ativado e a tecla premida for libertada. O diagrama temporal do controlo de fluxo está representado na Figura 2b.

O bloco *Key Scan* foi implementado de acordo com o diagrama de blocos representado na Figura 3.

A versão I deste diagrama foi selecionada tendo em consideração a sua simplicidade e facilidade de implementação. Apesar de não ser a mais eficiente — papel desempenhado pela versão III —, a escolha recaiu sobre a versão I por permitir obter resultados satisfatórios com um investimento reduzido de tempo e esforço. Esta decisão permitiu-nos concentrar mais recursos e atenção noutras componentes do projeto, de maior complexidade.

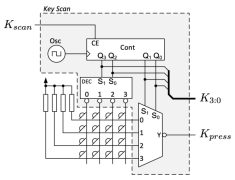


Figura 3: Diagrama de blocos do módulo *Key Scan*

O bloco *Key Control* foi implementado pela máquina de estados representada em *ASM-chart* na Figura 4. Esta estrutura permite detetar e validar de forma fiável a pressão de uma tecla proveniente do teclado matricial e é composta por quatro (4) estados: *INITIAL_STATE, KEY_PRESSED, WAIT_RELEASE e READY_NEXT.*

No estado *INITIAL_STATE*, o sistema ativa o sinal *Kscan*, responsável por iniciar o processo de varrimento do teclado matricial. Nesta fase, a máquina aguarda a ativação do sinal *Kpress*, que indica que uma tecla foi premida. Quando *Kpress* = 1, a máquina transita para o próximo estado (*KEY_PRESSED*).

No estado *KEY_PRESSED*, é ativado o sinal *Kval*, indicando que há uma tecla válida disponível para leitura. Este estado é mantido até que o sistema externo ative *Kack*, sinal que confirma que a tecla foi lida com sucesso. Com *Kack* = 1, a máquina avança para o estado seguinte (*WAIT_RELEASE*).

No estado *WAIT_RELEASE*, a máquina espera que *Kpress* seja desativado ($Kpress = 0$), ou seja, que a tecla deixe de estar pressionada. Este mecanismo garante que não são feitas leituras repetidas da mesma tecla enquanto estiver fisicamente premida. A transição para o estado *READY_NEXT* ocorre apenas quando, além de a tecla já ter sido libertada ($Kpress = 0$), o sinal de reconhecimento tiver igualmente sido desativado ($Kack = 0$), garantindo que o sistema externo concluiu o processamento da tecla anterior antes de permitir uma nova deteção.

A máquina chega ao estado *READY_NEXT* apenas depois de a tecla ter sido libertada e de o sinal *Kack* já estar a 0. Por isso, não há mais condições a verificar neste estado — a máquina volta de imediato ao estado *INITIAL_STATE* para iniciar a deteção de uma nova tecla.

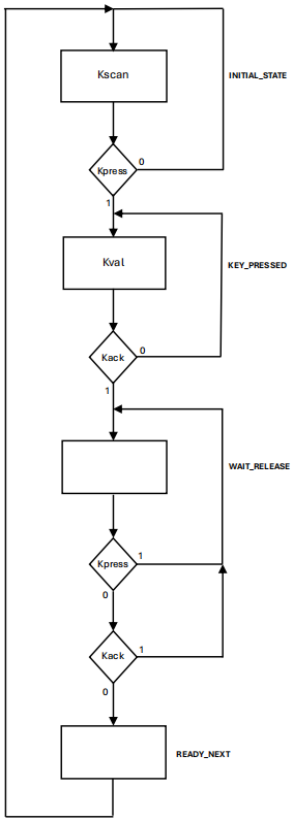


Figura 4: Máquina de estados do *Key Control*

A descrição hardware do bloco *Key Decode* em *VHDL* encontra-se no Anexo A.2.

2 Ring Buffer

O bloco *Ring Buffer* implementa uma estrutura de dados para armazenamento de teclas com disciplina FIFO (*First In First Out*), com capacidade de armazenar até oito palavras de quatro bits.

A escrita de dados no *Ring Buffer* inicia-se com a ativação do sinal *DAV* (*Data Available*) pelo sistema produtor, neste caso pelo *Key Decode*, indicando que tem dados para serem armazenados. Logo que tenha disponibilidade para armazenar informação, o *Ring Buffer* escreve os dados $D_{0:3}$ em memória. Concluída a escrita em memória ativa o sinal *DAC* (*Data Accepted*) para informar o sistema produtor que os dados foram aceites. O sistema produtor mantém o sinal *DAV* ativo até que *DAC* seja ativado. O *Ring Buffer* só desativa *DAC* depois de *DAV* ter sido desativado.

A implementação do *Ring Buffer* é baseada numa memória RAM (*Random Access Memory*). O endereço de escrita/leitura, selecionado por put(get), definido pelo bloco *Memory Address Control* (MAC) composto por dois registos, que contêm o endereço de escrita e leitura, designados por putIndex e getIndex respetivamente. O MAC suporta assim ações de incPut e incGet, gerando informação se a estrutura de dados está cheia (*Full*) ou se está vazia (*Empty*). O bloco *Ring Buffer* procede à entrega de dados à entidade consumidora, sempre que esta indique que está disponível para receber, através do sinal *Clear* To Send (CTS). Na Figura 5 é apresentado o diagrama de blocos para a estrutura do bloco *Ring Buffer*.

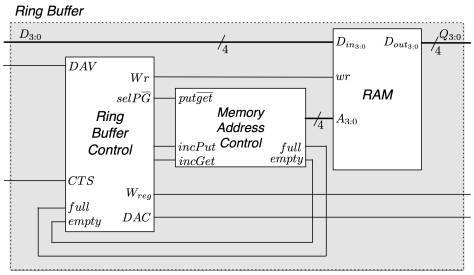


Figura 5: Diagrama de blocos do bloco *Ring Buffer*

2.1 Ring Buffer Control

O bloco *Ring Buffer Control* é responsável por gerir as operações de escrita e leitura no *Ring Buffer*, com base nos sinais de controlo *DAV* (dado disponível) e *CTS* (clear to send), e no estado de ocupação do buffer (*full* e *empty*).

A lógica do sistema é representada por uma máquina de estados sequencial (Figura 6), composta por sete estados:

INITIAL_STATE: estado de espera inicial, onde o sistema aguarda que *DAV* = '1' (novo dado disponível) ou *CTS* = '1' (permissão para leitura). Se *DAV* = '1' e o buffer estiver cheio (*full* = '1'), a máquina espera. Se *DAV* = '1' e *full* = '0', avança para escrita. Se *CTS* = '1' e *empty* = '0', avança para leitura.

ADDRESS_STATE: seleciona a operação a realizar (leitura ou escrita) através do sinal *selPG*, que indica se o acesso será por *put* ou *get*, e neste caso seria *put* (escrita)).

WRITE_STATE: se estiver a ser feita uma escrita, ativa-se o sinal *Wr* e mantém-se *selPG* a '0'. Este estado permite escrever os dados no endereço selecionado do buffer.

WNEXT_STATE: incrementa o contador *put* (com *incPut*) e mantém o *selPG*. A operação de escrita é considerada concluída.

DATAACCEPTED_STATE: ativa o sinal *DAC*, indicando que o dado foi aceite. O sistema espera que *DAV* = '0' (confirmação de que não há novo dado a enviar) para regressar ao estado inicial.

READ_STATE: ativa o sinal *Wreg*, permitindo ler os dados no endereço apontado pelo contador *get*. Esta leitura é feita quando *CTS* = '1' e o buffer não está vazio (*empty* = '0').

RNEXT_STATE: incrementa o contador *get* (com *incGet*) após confirmação (*CTS* = '0'), concluindo a operação de leitura e regressando ao estado inicial.

Esta lógica garante que apenas são realizados acessos válidos ao buffer: nunca se escreve quando está cheio, nem se lê quando está vazio. Além disso, assegura que os dados são processados uma única vez, sem duplicações ou perda de informação.

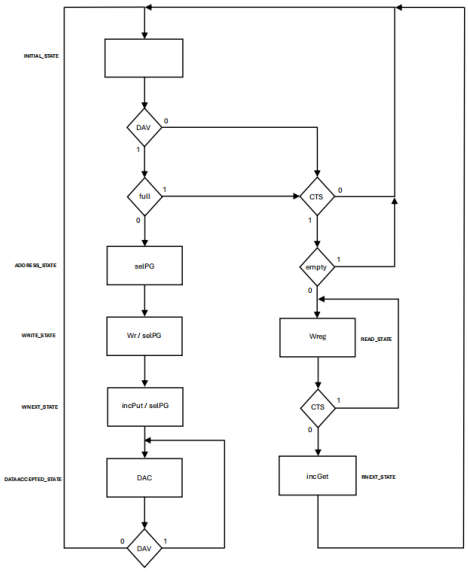


Figura 6: Máquina de estados do *Ring Buffer Control*

2.2 Memory Address Control

O bloco *MAC* (Figura 7) é responsável pela gestão dos endereços de leitura e escrita no *Ring Buffer*, garantindo que nunca se escreve em posições já ocupadas nem se lê de posições vazias. Para isso, o módulo gere separadamente os ponteiros de escrita (*put*) e de leitura (*get*), e calcula o estado do buffer com base num contador up/down.

A arquitetura do *MAC* integra:

Dois contadores simples (*counterPut* e *counterGet*), ambos com 4 bits, usados para gerar os endereços de escrita e leitura, respetivamente. Estes contadores são incrementados pelos sinais *incPut* e *incGet*.

Um multiplexer 2:1 (MUX2_1), que seleciona a saída do *counterPut* ou do *counterGet* para o endereço final A, consoante o sinal *putGet*. Assim, o endereço apresentado externamente depende da operação pretendida (leitura ou escrita).

Um contador up/down de 5 bits (CounterUD_MAC), responsável por controlar o número de posições ocupadas no buffer. O contador é ativado sempre que ocorre uma operação de leitura ou escrita, ou seja, quando *incPut* ou *incGet* estão ativos. A direção de contagem (*UpDown*) é controlada através da inversão do sinal *incGet*: se for '0', incrementa; se for '1', decrementa.

Dois comparadores (comparator_MAC) que comparam a saída do CounterUD_MAC com os valores "00000" e "11111". O primeiro define o sinal *empty*, que indica se o buffer está vazio, e o segundo define *full*, que indica que o buffer está cheio.

Esta lógica assegura que:

O endereço fornecido (A) é sempre válido e corresponde à operação em curso;

O estado do buffer (cheio ou vazio) é constantemente monitorizado;

O sistema impede sobreposição de dados ou leituras inválidas, promovendo uma gestão eficiente da memória circular.

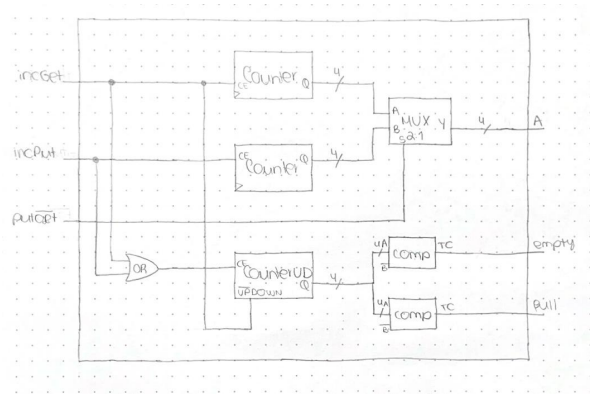


Figura 7: Diagrama de blocos do bloco *MAC*

3 Output Buffer

O bloco *Output Buffer* do *Keyboard Reader* é responsável pela interação com o sistema consumidor, neste caso o módulo *Control*. O *Output Buffer* indica que está disponível para armazenar dados através do sinal *OBfree*. Assim, nesta situação o sistema produtor pode ativar o sinal *Load* para registar os dados. O *Control* quando pretende ler dados do *Output Buffer*, aguarda que o sinal Dval fique ativo, recolhe os dados e ativa o sinal *ACK* indicando que estes já foram consumidos. O *Output Buffer*, logo que o sinal *ACK* seja ativado, deve invalidar os dados baixando o sinal *Dval* e sinalizar que está novamente disponível para entregar dados ao sistema consumidor, ativando o sinal *OBfree*. Na Figura 8, é apresentado o diagrama de blocos do *Output Buffer*. Sempre que o bloco emissor *Ring Buffer* tenha dados disponíveis e o bloco de entrega *Output Buffer* esteja disponível (*OBfree* ativo), o *Ring Buffer* realiza uma leitura da memória e entrega os dados ao *Output Buffer* ativando o sinal *Wreg*. O *Output Buffer* indica que já registou os dados desativando o sinal *OBfree*.

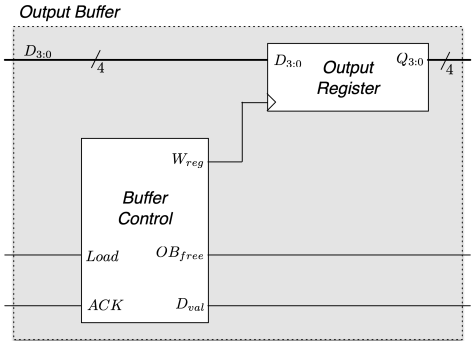


Figura 8: Diagrama de blocos do bloco *Output Buffer*

3.1 Buffer Control

O bloco *Buffer Control* coordena o envio de dados para o *Output Register*, garantindo que só são escritos novos dados quando o buffer estiver livre (*OBfree* = '1') e que os dados são mantidos até serem confirmados pelo sistema através do sinal *ACK*. O funcionamento é definido por uma máquina de estados sequencial (Figura **??**), composta por quatro estados principais:

INITIAL_STATE: estado de espera, onde o sistema monitoriza o sinal *Load*. Enquanto *Load* = '0', o bloco permanece neste estado e *OBfree* é mantido a '1', indicando que o buffer está disponível para receber dados.

LOAD_DATA: quando *Load* = '1', o bloco avança para este estado e ativa o sinal *Wreg*, permitindo carregar o valor de entrada no *Output Register*.

VALID_DATA: após o carregamento, é ativado o sinal *Dval*, que indica que o dado presente no registo é válido. O sistema aguarda neste estado até receber um *ACK* = '1', sinalizando que o dado foi lido pelo bloco seguinte.

WAIT_ACK0: após a leitura do dado, o sistema espera que o *ACK* volte a '0', garantindo que o próximo dado só será processado quando o an-

terior tiver sido completamente aceite. Só depois disso é que o sistema regressa ao INITIAL_STATE, reativando *OBfree*.

Esta lógica evita leituras múltiplas ou sobreposição de dados, garantindo que cada valor é enviado uma única vez e que o *Output Register* não é sobrecarregado enquanto não for confirmada a aceitação do dado anterior.

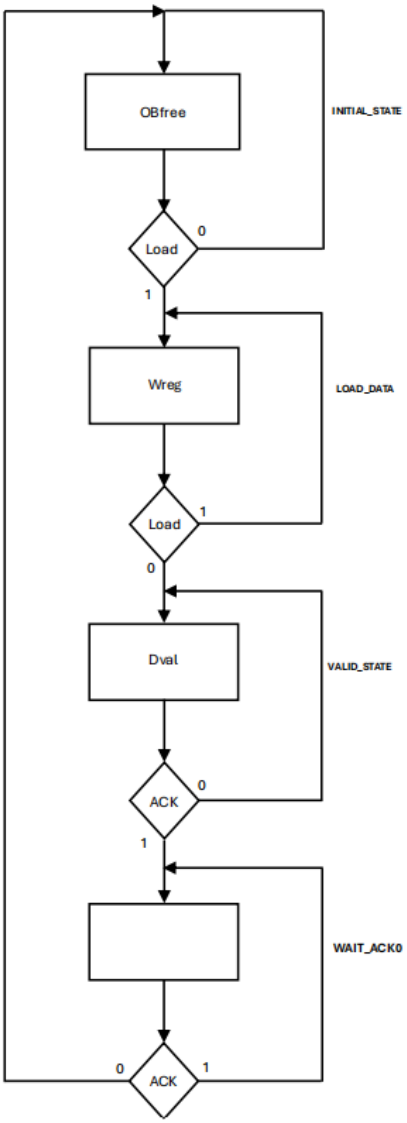


Figura 9: Máquina de estados do *Buffer Control*

4 Control

Implementou-se o módulo *Control* em *software*, recorrendo a linguagem *Kotlin* e seguindo a arquitetura lógica apresentada na Figura 10.

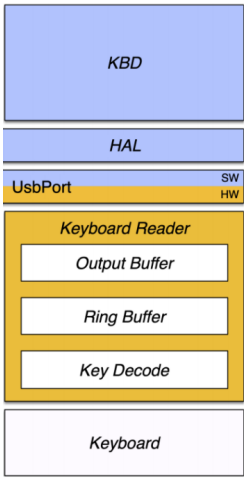


Figura 10: Diagrama lógico do módulo *Control* de interface com o módulo *Keyboard Reader*

O código-fonte das classes HAL e KBD foi desenvolvido nos Anexos B.1 e B.2, respetivamente.

4.1 Classe HAL

A classe HAL (*Hardware Abstraction Layer*) foi desenvolvida com o propósito de abstrair a comunicação direta com a porta USB da placa *d10lite*, facilitando a interação com os sinais digitais de entrada e saída do sistema. Esta camada de abstração permite ao programador manipular bits individuais ou conjuntos de bits de forma controlada e consistente, sem recorrer diretamente a operações de baixo nível.

Internamente, a classe mantém uma variável privada *currentState*, inicializada com o valor 0, que representa o estado atual dos bits de saída. Esta variável é atualizada a cada operação de escrita, garantindo que a informação enviada para a porta reflete sempre o estado mais recente dos sinais.

As principais funções implementadas incluem: **init()**: Inicializa o HAL escrevendo o valor inicial (0x00) na porta, garantindo um estado limpo no começo; **isBit(mask: Int)**: Verifica se um determinado bit está ativo (a '1') na leitura da porta;

readBits(mask: Int): Retorna os bits de entrada correspondentes à máscara fornecida; **setBits(mask: Int)** e **clrBits(mask: Int)**: Permitem, respetivamente, ativar ou limpar bits específicos, atualizando o estado global do output; **writeBits(mask: Int, value: Int)**: Escreve valores específicos nos bits definidos pela máscara, garantindo que apenas esses bits são alterados.

4.2 Classe KBD

A classe KBD (*Keyboard*) foi concebida para permitir a leitura eficiente e sincronizada de teclas a partir do teclado matricial presente na placa. O mapeamento das teclas (0 a 9, A a D, * e #) é realizado com base no vetor *TECLADO*, que associa os valores binários lidos aos respetivos caracteres.

Esta classe tira partido dos sinais *DVAL* (indicador de que uma tecla foi premida) e *ACK* (confirmação de leitura), utilizando os métodos da classe HAL para interagir com os sinais digitais de entrada/saída.

As funções desenvolvidas são: **init()**: Inicializa a classe chamando o método init() da HAL; **getKey()**: Verifica se existe uma tecla premida. Caso exista, lê os bits correspondentes, traduz a tecla com base no vetor *TECLADO*, ativa o sinal *ACK* para assinalar a leitura e espera que a tecla seja libertada antes de limpar o *ACK*; **waitKey(timeout: Long)**: Aguarda por uma tecla durante um período de tempo especificado. Se nenhuma tecla for detetada dentro do intervalo definido, retorna o valor *NONE*.

5 Conclusões

O desenvolvimento do módulo *Keyboard Reader* permitiu implementar uma interface funcional e fiável entre o teclado matricial e o sistema consumidor, assegurando uma leitura síncrona das teclas e o armazenamento temporário dos dados com disciplina FIFO.

A arquitetura modular, composta pelos blocos *Key Decode*, *Ring Buffer* e *Output Buffer*, garantiu uma separação clara de responsabilidades: desde a deteção da tecla, à gestão da memória circular e à entrega controlada dos dados ao sistema.

O *Ring Buffer*, implementado com controlo de escrita/leitura e deteção de estados *full* e *empty*, permitiu o armazenamento eficiente das teclas, sem perdas nem sobreposições. Por sua vez, o *Output Buffer* assegurou que os dados fossem enviados apenas após confirmação, evitando o envio prematuro ou repetido.

A integração com o software, através da classe *KBD* em Kotlin, demonstrou a eficácia da comunicação entre o sistema físico e a interface de alto nível, permitindo ler e interpretar as teclas com fiabilidade e facilidade de uso.

Este projeto demonstrou a importância da coordenação entre hardware e software, bem como a utilidade de máquinas de estados e buffers no tratamento de entrada de dados em sistemas digitais.

A VHDL

A.1 Keyboard Reader

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyboardReader is
        port(
        RESET : in std_logic;
        CLK : in std_logic;
        Dval : out std_logic;
        D : out std_logic_vector (3 downto 0);
        KeyPadL : in std_logic_vector (3 downto 0);
        KeyPadC : out std_logic_vector (3 downto 0);
        ACK : in std_logic
        );
end KeyboardReader;

architecture structural of KeyboardReader is

component KeyDecode is
        port(
        Kack : in std_logic;
        Kval : out std_logic;
        RESET : in std_logic;
        CLK : in std_logic;
        K : out std_logic_vector (3 downto 0);
        KeyPadL : in std_logic_vector (3 downto 0);
        KeyPadC : out std_logic_vector (3 downto 0)
        );
end component;

component RingBuffer is
        port(
        D : in std_logic_vector(3 downto 0);
        DAV : in std_logic;
        CTS : in std_logic;
        RESET : in std_logic;
        CLK : in std_logic;
        Wreg : out std_logic;
        DAC : out std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

component OutputBuffer is
        port(
        D : in std_logic_vector(3 downto 0);
        Load : in std_logic;
        ACK : in std_logic;
        RESET : in std_logic;
        CLK : in std_logic;
        OBfree : out std_logic;
        Dval : out std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

signal Kval_DAV : std_logic;
signal K_D : std_logic_vector(3 downto 0);
signal Q_D : std_logic_vector(3 downto 0);
```

```
signal Wreg_Load : std_logic;
signal OBfree_CTS : std_logic;
signal DAC_Kack : std_logic;
signal Dval_exit : std_logic;
signal Dexit : std_logic_vector(3 downto 0);

begin
U0: KeyDecode port map (
        CLK => CLK,
        RESET => RESET,
        Kval => Kval_DAV,
        K => K_D,
        Kack => DAC_Kack,
        KeyPadL(0) => KeyPadL(0),
        KeyPadL(1) => KeyPadL(1),
        KeyPadL(2) => KeyPadL(2),
        KeyPadL(3) => KeyPadL(3),
        KeyPadC(0) => KeyPadC(0),
        KeyPadC(1) => KeyPadC(1),
        KeyPadC(2) => KeyPadC(2),
        KeyPadC(3) => KeyPadC(3)
        );

U1: RingBuffer port map (
        D => K_D,
        DAV => Kval_DAV,
        CTS => OBfree_CTS,
        RESET => RESET,
        CLK => CLK,
        Wreg => Wreg_Load,
        DAC => DAC_Kack,
        Q => Q_D
        );

U2: OutputBuffer port map (
        D => Q_D,
        Load => Wreg_Load,
        ACK => ACK,
        RESET => RESET,
        CLK => CLK,
        OBfree => OBfree_CTS,
        Dval => Dval_exit,
        Q => Dexit
        );

D <= Dexit;
Dval <= Dval_exit;

end structural;
```

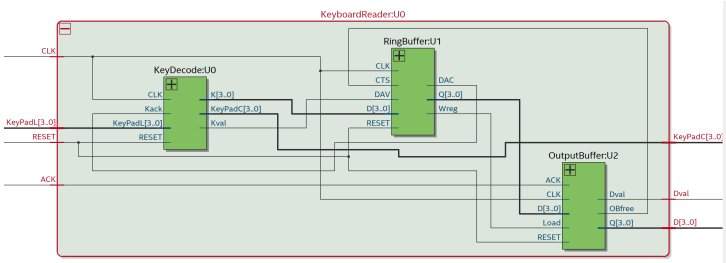


Figura 11: Netlist Viewer do *Keyboard Reader*

A.2 Key Decode

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyDecode is
        port(
        Kack : in std_logic;
        Kval : out std_logic;
        RESET : in std_logic;
        CLK : in std_logic;
        K : out std_logic_vector (3 downto 0);
        KeyPadL : in std_logic_vector (3 downto 0);
        KeyPadC : out std_logic_vector (3 downto 0)
        );
end KeyDecode;

architecture structural of KeyDecode is

component KeyScan is
        port(
        Kscan : in std_logic;
        RESET : in std_logic;
        CLK : in std_logic;
        inMUX : in std_logic_vector (3 downto 0);
        K : out std_logic_vector (3 downto 0);
        Kpress : out std_logic;
        outDEC : out std_logic_vector(3 downto 0)
        );
end component;

component KeyControl is
        port(
        Kack : in std_logic;
        Kpress : in std_logic;
   RESET : in std_logic;
   CLK : in std_logic;
   Kval : out std_logic;
        Kscan : out std_logic
        );
end component;

component CLKDIV is
        port (
        clk_in: in std_logic;
        clk_out: out std_logic
        );
end component;
```

```
signal Kexit : std_logic_vector(3 downto 0);
signal Kpress_Kpress : std_logic;
signal Kscan_Kscan : std_logic;
signal Sclk : std_logic;

begin
U0: KeyScan port map ( Kscan => Kscan_Kscan, CLK => Sclk, RESET => RESET, inMUX(0) => KeyPadL(0),
inMUX(1) => KeyPadL(1), inMUX(2) => KeyPadL(2), inMUX(3) => KeyPadL(3), K => Kexit, Kpress => Kpress_Kpress,
outDEC(0) => KeyPadC(0), outDEC(1) => KeyPadC(1), outDEC(2) => KeyPadC(2), outDEC(3) => KeyPadC(3) );

U1: KeyControl port map( Kack => Kack, Kpress => Kpress_Kpress, Kval => Kval, Kscan => Kscan_Kscan,
CLK => not Sclk, RESET => RESET );

U2: CLKDIV port map( clk_in => CLK, clk_out => Sclk );

K <= Kexit;

end structural;
```

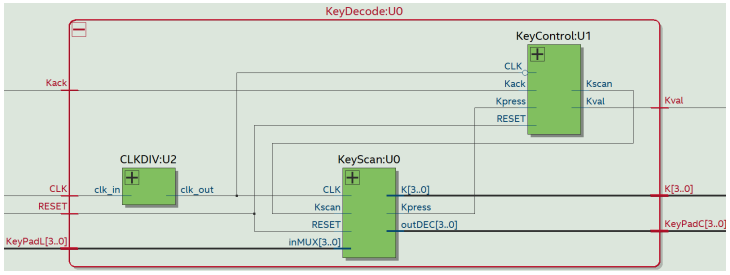


Figura 12: Netlist Viewer do *Key Decode*

A.3 CLKDIV

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity CLKDIV is
generic(div: natural := 500000);
        port (
        clk_in: in std_logic;
        clk_out: out std_logic
        );
end CLKDIV;

architecture bhv of CLKDIV is

signal count: integer:= 1;
signal tmp : std_logic := '0';

begin

process(clk_in)

begin

if(clk_in'event and clk_in ='1') then
count <= count + 1;
if (count = div/2) then
tmp <= NOT tmp;
```

```
count <= 1;
end if;
end if;
end process;

clk_out <= tmp;

end bhv;
```

A.4 Key Control

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyControl is
        port(
        Kack : in std_logic;
        Kpress : in std_logic;
   RESET : in std_logic;
   CLK : in std_logic;
   Kval : out std_logic;
        Kscan : out std_logic
        );
end KeyControl;

architecture behavioral of KeyControl is

type STATE_TYPE is (INITIAL_STATE, KEY_PRESSED, WAIT_RELEASE, READY_NEXT);

signal CurrentState, NextState : STATE_TYPE;

begin

CurrentState <= INITIAL_STATE when RESET = '1' else NextState when rising_edge(CLK);

GenerateNextState:
process (CurrentState, Kpress , Kack)
        begin
                case CurrentState is
                        when INITIAL_STATE        => if (Kpress = '1') then
                                                                                                                NextState <= KEY_PRESSED;
                                                                                                        else
                                                                                                                NextState <= INITIAL_STATE;
                                                                                                        end if;

                        when KEY_PRESSED                => if (Kack = '1') then
                                                                                                                NextState <= WAIT_RELEASE;
                                                                                                        else
                                                                                                                NextState <= KEY_PRESSED;
                                                                                                        end if;

                        when WAIT_RELEASE                => if (Kpress = '0' and Kack = '0') then
                                                                                                                NextState <= READY_NEXT;
                                                                                                        else
                                                                                                                NextState <= WAIT_RELEASE;
                                                                                                        end if;

                        when READY_NEXT                => NextState <= INITIAL_STATE;
                end case;
        end process;

Kscan <= '1' when (CurrentState = INITIAL_STATE) else '0';
```

```
Kval <= '1' when (CurrentState = KEY_PRESSED) else '0';

end behavioral;
```

A.5 Key Scan

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyScan is
        port(
        Kscan : in std_logic;
        RESET : in std_logic;
        CLK : in std_logic;
        inMUX : in std_logic_vector (3 downto 0);
        K : out std_logic_vector (3 downto 0);
        Kpress : out std_logic;
        outDEC : out std_logic_vector(3 downto 0)
        );
end KeyScan;

architecture structural of KeyScan is

component Counter is
        port (
        CE,CLK,RESET : in std_logic;
        Q: out std_logic_vector (3 downto 0)
        );
end component;

component DEC2_4 is
        port(
        S0, S1: in std_logic;
        A, B, C, D: out std_logic
        );
end component;

component MUX4_1 is
        port(
    A, B, C, D : in std_logic;
    S0, S1 : in std_logic;
    Y : out std_logic
    );
end component;

signal exitMUX: std_logic;
signal q: std_logic_vector (3 downto 0);

begin

U0: Counter port map ( CE => Kscan , CLK => CLK , RESET => RESET , Q => q );
U1: DEC2_4 port map ( S1 => q(3), S0 => q(2), A => outDEC(0), B => outDEC(1), C => outDEC(2),
D => outDEC(3) );
U2: MUX4_1 port map ( A => inMUX(0) , B => inMux(1), C => inMUx(2) , D => inMux(3), S1 => q(1), S0 => q(0), Y => exitMUX );
K <= q;
Kpress <= exitMUX;

end structural;
```

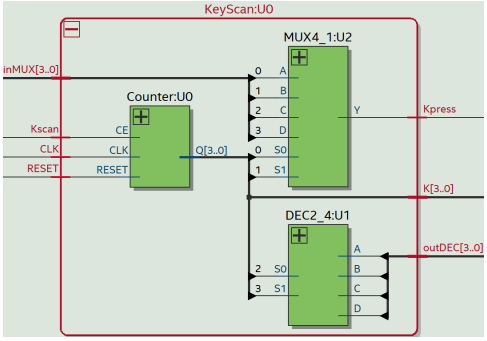


Figura 13: Netlist Viewer do *Key Scan*

A.6 Counter

```
library ieee;
use ieee.std_logic_1164.all;

entity Counter is
        port (
        CE,CLK,RESET : in std_logic ;
        Q: out std_logic_vector (3 downto 0)
        );
end Counter;

architecture structural of Counter is

component Adder is
        port(
   A, B: in std_logic_vector(3 downto 0);
   Ci: in std_logic;
   Co: out std_logic;
   S: out std_logic_vector(3 downto 0)
   );
end component;

component Reg is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

signal Adder_Reg: std_logic_vector (3 downto 0);
signal Reg_exit: std_logic_vector (3 downto 0);

begin

U0: Adder port map (A => Reg_exit, B => "0001", Ci => '0', Co => open, S => Adder_Reg );
U1: Reg port map (CLK => CLK, RESET => RESET, D => Adder_Reg, EN => CE, Q => Reg_exit);
Q <= Reg_exit;

end structural;
```

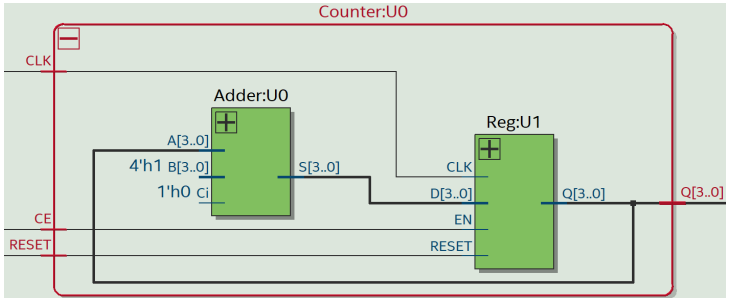


Figura 14: Netlist Viewer do *Counter*

A.7 Register

```
library ieee;
use ieee.std_logic_1164.all;

entity Reg is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end Reg;

architecture structural of Reg is

component FFD_KS is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN : in std_logic;
        Q : out std_logic
        );
end component;

begin

U0: FFD_KS port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
U1: FFD_KS port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
U2: FFD_KS port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));
U3: FFD_KS port map(CLK => CLK, RESET => RESET, SET => '0', D => D(3), EN => EN, Q => Q(3));

end structural;
```

A.8 Adder

```
library ieee;
use ieee.std_logic_1164.all;

entity Adder is
        port(
        A, B: in std_logic_vector(3 downto 0);
        Ci: in std_logic;
        S: out std_logic_vector(3 downto 0);
```

```
        Co: out std_logic
        );
end Adder;

architecture structural of Adder is

component FA is
        port(
        A, B, Ci : in std_logic;
        S, Co: out std_logic
        );
end component;

signal carry: std_logic_vector(3 downto 1);

begin

U0: FA port map (A => A(0), B => B(0), Ci => Ci, S => S(0), Co => carry(1));
U1: FA port map (A => A(1), B => B(1), Ci => carry(1), S => S(1), Co => carry(2));
U2: FA port map (A => A(2), B => B(2), Ci => carry(2), S => S(2), Co => carry(3));
U3: FA port map (A => A(3), B => B(3), Ci => carry(3), S => S(3), Co => Co);

end structural;
```

A.9 Full Adder

```
library ieee;
use ieee.std_logic_1164.all;

entity FA is
        port(
   A, B, Ci: in std_logic;
   S, Co: out std_logic
   );
end FA;

architecture structural of FA is

component HA is
    port(
    A, B: in std_logic;
    S, Co: out std_logic
    );
end component;

signal addAB: std_logic;
signal carry: std_logic_vector(1 downto 0);

begin

U1: HA port map (A => A, B => B, S => addAB, Co => carry(0));
U2: HA port map (A => addAB, B => Ci, S => S, Co => carry(1));

Co <= carry(0) or carry(1);


end structural;
```

A.10 Half Adder

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity HA is
        port(
        A, B: in std_logic;
        S, Co: out std_logic
        );
end HA;

architecture structural of HA is
begin

S <= A xor B;
Co <= A and B;

end structural;
```

A.11 MUX

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX4_1 is
    port(
    A, B, C, D : in std_logic;
    S0, S1 : in std_logic;
    Y : out std_logic
    );
end MUX4_1;

architecture structural of MUX4_1 is

begin

Y <= not ((A and not S1 and not S0) or (B and not S1 and S0) or (C and S1 and not S0) or (D and S1 and S0));

end structural;
```

A.12 Decoder

```
library ieee;
use ieee.std_logic_1164.all;

entity DEC2_4 is
        port(
        S0, S1: in std_logic;
        A, B, C, D: out std_logic
        );
end DEC2_4;

architecture structural of DEC2_4 is
begin

A <= not(not S1 and not S0);
B <= not(not S1 and S0);
C <= not(S1 and not S0);
D <= not(S1 and S0);

end structural;
```

A.13 Flip-Flop

```
library ieee;
use ieee.std_logic_1164.all;

entity FFD_KS is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end FFD_KS;

architecture structural of FFD_KS is

begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

A.14 RingBuffer

```
library ieee;
use ieee.std_logic_1164.all;

entity RingBuffer is
        port(
        D : in std_logic_vector(3 downto 0);
        DAV : in std_logic;
        CTS : in std_logic;
        RESET : in std_logic;
        CLK : in std_logic;
        Wreg : out std_logic;
        DAC : out std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end RingBuffer;

architecture structural of RingBuffer is

component MAC is
        port(
        putGet: in std_logic;
        incPut: in std_logic;
        incGet: in std_logic;
        CLK: in std_logic;
        RESET: in std_logic;
        full: out std_logic;
        empty: out std_logic;
        A: out std_logic_vector(3 downto 0)
        );
end component;

component RingBuffer_Control is
        port(
        DAV : in std_logic;
        CLK : in std_logic;
        RESET : in std_logic;
```

```
        CTS : in std_logic;
        full : in std_logic;
        empty : in std_logic;
        Wr : out std_logic;
        selPG : out std_logic;
        Wreg : out std_logic;
        DAC : out std_logic;
        incPut : out std_logic;
        incGet : out std_logic
        );
end component;

component RAM is
        generic(
                ADDRESS_WIDTH : natural := 4;
                DATA_WIDTH : natural := 4
        );
        port(
                address : in std_logic_vector(ADDRESS_WIDTH - 1 downto 0);
                wr: in std_logic;
                din: in std_logic_vector(DATA_WIDTH - 1 downto 0);
                dout: out std_logic_vector(DATA_WIDTH - 1 downto 0)
        );
end component;

signal wr_signal: std_logic;
signal selPG_putget: std_logic;
signal MAC_A: std_logic_vector(3 downto 0);
signal incGet_signal: std_logic;
signal incPut_signal: std_logic;
signal full_signal: std_logic;
signal empty_signal: std_logic;

begin

U0 : MAC port map (
        putGet => selPG_putget,
        incPut => incPut_signal,
        incGet => incGet_signal,
        CLK => CLK,
        RESET => RESET,
        full => full_signal,
        empty => empty_signal,
        A => MAC_A
        );

U1 : RingBuffer_Control port map (
        DAV => DAV,
        CLK => CLK,
        RESET => RESET,
        CTS => CTS,
        full => full_signal,
        empty => empty_signal,
        Wr => wr_signal,
        selPG => selPG_putget,
        Wreg => Wreg,
        DAC => DAC,
        incPut => incPut_signal,
        incGet => incGet_signal
        );

U2 : RAM port map (
```

```
        address => MAC_A,
        wr => wr_signal,
        din => D,
        dout => Q
        );

end structural;
```

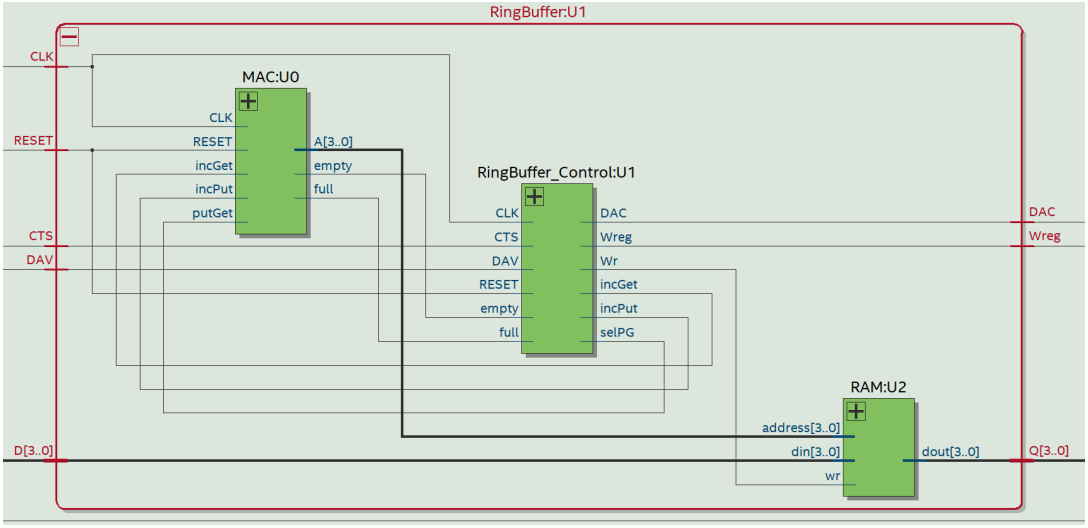


Figura 15: Netlist Viewer do *RingBuffer*

A.15 MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity MAC is
        port(
        putGet: in std_logic;
        incPut: in std_logic;
        incGet: in std_logic;
        CLK: in std_logic;
        RESET: in std_logic;
        full: out std_logic;
        empty: out std_logic;
        A: out std_logic_vector(3 downto 0)
        );
end MAC;

architecture structural of MAC is

component CounterUD_MAC is
        port (
        CE,CLK,RESET, UP_DOWN: in std_logic;
        Q: out std_logic_vector (4 downto 0)
        );
end component;

component Counter_MAC is
        port (
        CE,CLK,RESET : in std_logic ;
```

```
        Q: out std_logic_vector (3 downto 0)
        );
end component;

component MUX2_1 is
        port (
        A, B : in std_logic_vector(3 downto 0);
        S: in std_logic;
   Y : out std_logic_vector (3 downto 0)
   );
end component;

component comparator_MAC is
        port (
        A: in std_logic_vector (4 downto 0);
        B: in std_logic_vector (4 downto 0);
        TC: out std_logic
        );
end component;

signal getCounter_mux : std_logic_vector(3 downto 0);
signal putCounter_mux : std_logic_vector(3 downto 0);
signal CounterUD_cmp : std_logic_vector(4 downto 0);
signal or_signal : std_logic;

begin
U0: Counter_MAC port map ( CE => incGet, CLK => CLK, RESET => RESET, Q => getCounter_mux );

U1: Counter_MAC port map ( CE => incPut, CLK => CLK, RESET => RESET, Q => putCounter_mux );

U2: CounterUD_MAC port map ( CE => or_signal, CLK => CLK, RESET => RESET, UP_DOWN => incGet, Q => CounterUD_cmp );

U3: MUX2_1 port map ( A => getCounter_mux, B => putCounter_mux, S => putGet, Y => A );

U4: comparator_MAC port map ( A => CounterUD_cmp, B => "00000", TC => empty );

U5: comparator_MAC port map ( A => CounterUD_cmp, B => "10000", TC => full );

or_signal <= incPut or incGet;

end structural;
```

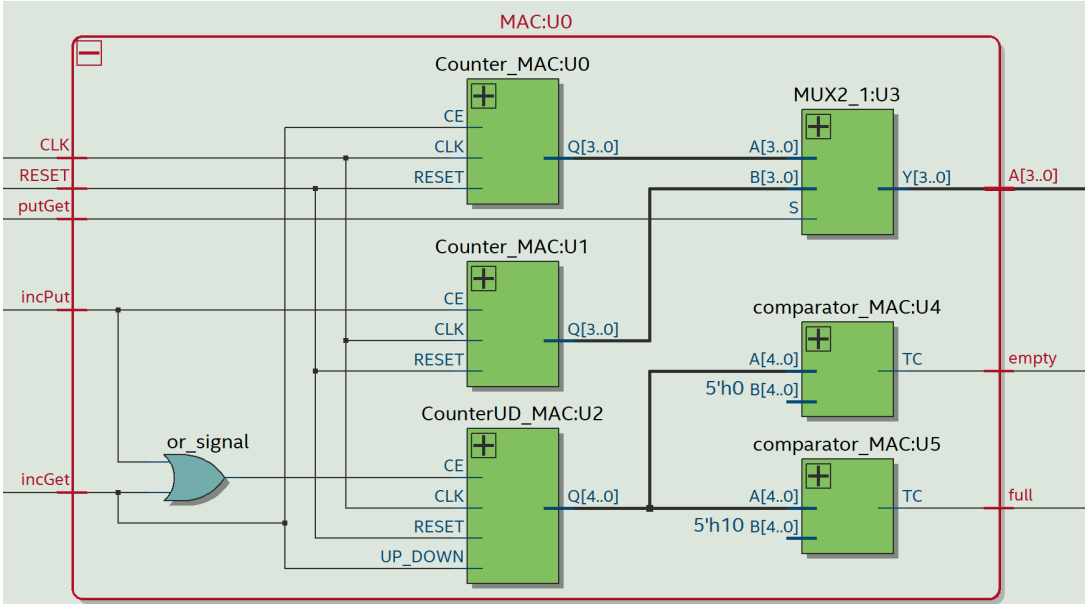


Figura 16: Netlist Viewer do *MAC*

A.16 Counter_MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity Counter_MAC is
        port (
        CE,CLK,RESET : in std_logic ;
        Q: out std_logic_vector (3 downto 0)
        );
end Counter_MAC;

architecture structural of Counter_MAC is

component Adder_MAC is
        port(
   A, B: in std_logic_vector(3 downto 0);
   Ci: in std_logic;
   Co: out std_logic;
   S: out std_logic_vector(3 downto 0)
   );
end component;

component Reg_MAC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

signal Adder_Reg: std_logic_vector (3 downto 0);
signal Reg_exit: std_logic_vector (3 downto 0);
```

```
begin

U0: Adder_MAC port map ( A => Reg_exit, B => "0001", Ci => '0', Co => open, S => Adder_Reg );
U1: Reg_MAC port map ( CLK => CLK, RESET => RESET, D => Adder_Reg, EN => CE, Q => Reg_exit );
Q <= Reg_exit;

end structural;
```

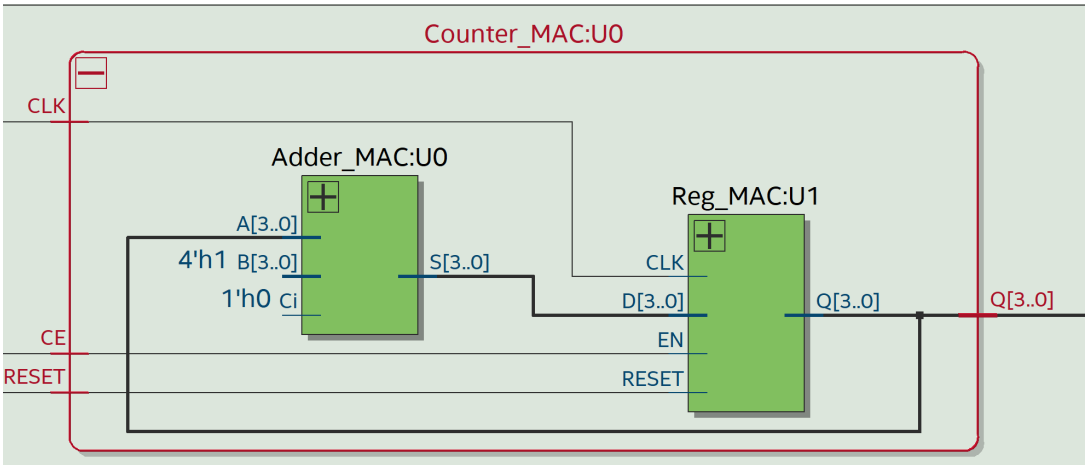


Figura 17: Netlist Viewer do *Counter_MAC*

A.17 Adder_MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity Adder_MAC is
        port(
        A, B: in std_logic_vector(3 downto 0);
        Ci: in std_logic;
        S: out std_logic_vector(3 downto 0);
        Co: out std_logic
        );
end Adder_MAC;

architecture structural of Adder_MAC is

component FA_MAC is
        port(
        A, B, Ci : in std_logic;
        S, Co: out std_logic
        );
end component;

signal carry: std_logic_vector(3 downto 1);

begin

U0: FA_MAC port map (A => A(0), B => B(0), Ci => Ci, S => S(0), Co => carry(1));
U1: FA_MAC port map (A => A(1), B => B(1), Ci => carry(1), S => S(1), Co => carry(2));
U2: FA_MAC port map (A => A(2), B => B(2), Ci => carry(2), S => S(2), Co => carry(3));
U3: FA_MAC port map (A => A(3), B => B(3), Ci => carry(3), S => S(3), Co => Co);
```

```
end structural;
```

A.18 FA_MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity FA_MAC is
        port(
   A, B, Ci: in std_logic;
   S, Co: out std_logic
   );
end FA_MAC;

architecture structural of FA_MAC is

component HA_MAC is
    port(
    A, B: in std_logic;
    S, Co: out std_logic
    );
end component;

signal addAB: std_logic;
signal carry: std_logic_vector(1 downto 0);

begin

U1: HA_MAC port map (A => A, B => B, S => addAB, Co => carry(0));
U2: HA_MAC port map (A => addAB, B => Ci, S => S, Co => carry(1));

Co <= carry(0) or carry(1);


end structural;
```

A.19 HA_MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity HA_MAC is
        port(
        A, B: in std_logic;
        S, Co: out std_logic
        );
end HA_MAC;

architecture structural of HA_MAC is
begin

S <= A xor B;
Co <= A and B;

end structural;
```

A.20 Reg_MAC

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity Reg_MAC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end Reg_MAC;

architecture structural of Reg_MAC is

component FFD_MAC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN : in std_logic;
        Q : out std_logic
        );
end component;

begin

U0: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
U1: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
U2: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));
U3: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(3), EN => EN, Q => Q(3));

end structural;
```

A.21 FFD_MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity FFD_MAC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end FFD_MAC;

architecture structural of FFD_MAC is

begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

A.22 CounterUD_MAC

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity CounterUD_MAC is
    port (
        CE, CLK, RESET, UP_DOWN : in std_logic;
        Q : out std_logic_vector(4 downto 0)
    );
end CounterUD_MAC;

architecture structural of CounterUD_MAC is

component Adder5_MAC is
        port(
   A, B : in std_logic_vector(4 downto 0);
   Ci : in std_logic;
   Co : out std_logic;
   S : out std_logic_vector(4 downto 0)
   );
end component;

component Reg5_MAC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(4 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(4 downto 0)
   );
end component;

component MUX5_2_1 is
        port(
        A, B : in std_logic_vector(4 downto 0);
        S : in std_logic;
        Y : out std_logic_vector(4 downto 0)
        );
end component;

signal Adder_Reg, Reg_exit, B_mux : std_logic_vector(4 downto 0);

begin
U0: MUX5_2_1 port map(
        A => "00001",
        B => "11111",
        S => UP_DOWN,
        Y => B_mux
                  );

U1: Adder5_MAC port map(
        A => Reg_exit,
        B => B_mux,
        Ci => '0',
        Co => open,
        S => Adder_Reg
        );

U2: Reg5_MAC port map(
        CLK => CLK,
        RESET => RESET,
        D => Adder_Reg,
        EN => CE,
        Q => Reg_exit
   );
```

```
Q <= Reg_exit;

end structural;
```

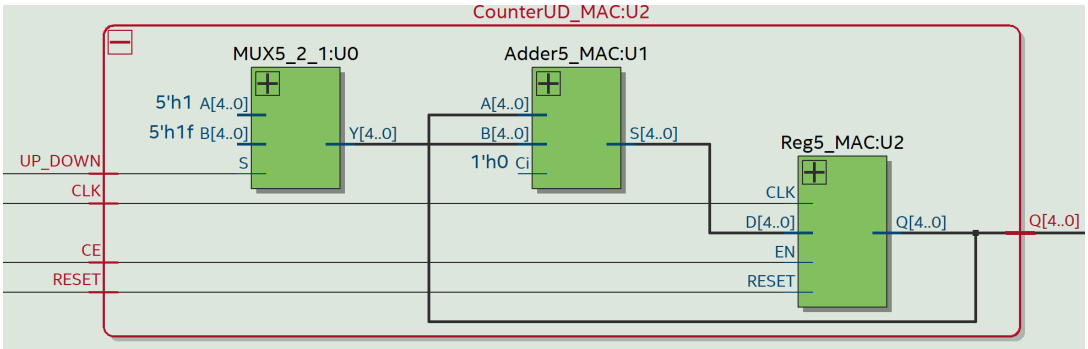


Figura 18: Netlist Viewer do *CounterUD_MAC*

A.23 MUX5_2_1

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX5_2_1 is
        port (
        A, B : in std_logic_vector(4 downto 0);
        S: in std_logic;
   Y : out std_logic_vector (4 downto 0)
   );
end MUX5_2_1;

architecture structural of MUX5_2_1 is
begin

Y(0) <= (A(0) and not S) or (B(0) and S);
Y(1) <= (A(1) and not S) or (B(1) and S);
Y(2) <= (A(2) and not S) or (B(2) and S);
Y(3) <= (A(3) and not S) or (B(3) and S);
Y(4) <= (A(4) and not S) or (B(4) and S);

end structural;
```

A.24 Adder5_MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity Adder5_MAC is
        port(
        A, B: in std_logic_vector(4 downto 0);
        Ci: in std_logic;
        S: out std_logic_vector(4 downto 0);
        Co: out std_logic
        );
end Adder5_MAC;

architecture structural of Adder5_MAC is
```

```
component FA_MAC is
        port(
        A, B, Ci : in std_logic;
        S, Co: out std_logic
        );
end component;

signal carry: std_logic_vector(4 downto 1);

begin

U0: FA_MAC port map (A => A(0), B => B(0), Ci => Ci, S => S(0), Co => carry(1));
U1: FA_MAC port map (A => A(1), B => B(1), Ci => carry(1), S => S(1), Co => carry(2));
U2: FA_MAC port map (A => A(2), B => B(2), Ci => carry(2), S => S(2), Co => carry(3));
U3: FA_MAC port map (A => A(3), B => B(3), Ci => carry(3), S => S(3), Co => carry(4));
U4: FA_MAC port map (A => A(4), B => B(4), Ci => carry(4), S => S(4), Co => Co);

end structural;
```

A.25 Reg5_MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity Reg5_MAC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(4 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(4 downto 0)
        );
end Reg5_MAC;

architecture structural of Reg5_MAC is

component FFD_MAC is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN : in std_logic;
        Q : out std_logic
        );
end component;

begin

U0: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
U1: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
U2: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));
U3: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(3), EN => EN, Q => Q(3));
U4: FFD_MAC port map(CLK => CLK, RESET => RESET, SET => '0', D => D(4), EN => EN, Q => Q(4));

end structural;
```

A.26 MUX2_1

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity MUX2_1 is
        port (
        A, B : in std_logic_vector(3 downto 0);
        S: in std_logic;
   Y : out std_logic_vector (3 downto 0)
   );
end MUX2_1;

architecture structural of MUX2_1 is
begin

Y(0) <= (A(0) and not S) or (B(0) and S);
Y(1) <= (A(1) and not S) or (B(1) and S);
Y(2) <= (A(2) and not S) or (B(2) and S);
Y(3) <= (A(3) and not S) or (B(3) and S);

end structural;
```

A.27 comparator_MAC

```
library ieee;
use ieee.std_logic_1164.all;

entity comparator_MAC is
        port (
        A: in std_logic_vector (4 downto 0);
        B: in std_logic_vector (4 downto 0);
        TC: out std_logic
        );
end comparator_MAC;

architecture structural of comparator_MAC is
begin

TC <= ((A(0) xnor B(0)) and (A(1) xnor B(1)) and (A(2) xnor B(2)) and (A(3) xnor B(3)) and (A(4) xnor B(4)) );

end structural;
```

A.28 RingBuffer_Control

```
library ieee;
use ieee.std_logic_1164.all;

entity RingBuffer_Control is
        port(
        DAV : in std_logic;
        CLK : in std_logic;
        RESET : in std_logic;
        CTS : in std_logic;
        full : in std_logic;
        empty : in std_logic;
        Wr : out std_logic;
        selPG : out std_logic;
        Wreg : out std_logic;
        DAC : out std_logic;
        incPut : out std_logic;
        incGet : out std_logic
        );
end RingBuffer_Control;

architecture behavioral of RingBuffer_Control is
```

```
type STATE_TYPE is (INITIAL_STATE, ADDRESS_STATE, WRITE_STATE, WNEXT_STATE, DATAACCEPTED_STATE, READ_STATE, RNEXT_STATE);

signal Current_State, Next_State : STATE_TYPE;

begin

Current_State <= INITIAL_STATE when RESET = '1' else Next_State when rising_edge(clk);

GenerateNextState:
process (Current_State, DAV, CTS, full, empty)
        begin
                case Current_State is
                        when INITIAL_STATE => if (DAV = '1' and full = '0') then
                                                Next_State <= ADDRESS_STATE;
                                              elsif (DAV = '1' and full = '1' and CTS = '1' and empty = '0') then
                                                Next_State <= READ_STATE;
                                              elsif (DAV = '0' and CTS = '1' and empty = '0') then
                                                Next_State <= READ_STATE;
                                              else
                                                Next_State <= INITIAL_STATE;
                                              end if;

                when ADDRESS_STATE => Next_State <= WRITE_STATE;

                when WRITE_STATE => Next_State <= WNEXT_STATE;

                when WNEXT_STATE => Next_State <= DATAACCEPTED_STATE;

                when DATAACCEPTED_STATE => if (DAV = '1') then
                                                Next_State <= DATAACCEPTED_STATE;
                                           else
                                                Next_State <= INITIAL_STATE;
                                           end if;

                when READ_STATE => if (CTS = '1') then
                                        Next_State <= READ_STATE;
                                   else
                                        Next_State <= RNEXT_STATE;
                                   end if;

                when RNEXT_STATE => Next_State <= INITIAL_STATE;



        end case;
end process;

selPG <= '1' when (Current_State = ADDRESS_STATE or Current_State = WRITE_STATE or Current_State = WNEXT_STATE) else '0';

Wr <= '1' when (Current_State = WRITE_STATE) else '0';

incPut <= '1' when (Current_State = WNEXT_STATE) else '0';

DAC <= '1' when (Current_State = DATAACCEPTED_STATE) else '0';

Wreg <= '1' when (Current_State = READ_STATE) else '0';

incGet <= '1' when (Current_State = RNEXT_STATE) else '0';

end behavioral;
```

A.29 RAM

```
-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
-- Project     : DE10-Lite
-- Affiliations: DEETC, ISEL - IPL
-- Funding     : -
-------------------------------------------------------------------------------
-- File        : RAM.vhd
-- Author(s)   : Pedro Miguens Matutino
-- Date        : 2023/01/03
-------------------------------------------------------------------------------
-- Copyright (c) 2023 Pedro Miguens Matutino
-------------------------------------------------------------------------------
-- Description :
-- .
-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;

entity RAM is
        generic(
                ADDRESS_WIDTH : natural := 4;
                DATA_WIDTH : natural := 4
        );
        port(
                address : in std_logic_vector(ADDRESS_WIDTH - 1 downto 0);
                wr: in std_logic;
                din: in std_logic_vector(DATA_WIDTH - 1 downto 0);
                dout: out std_logic_vector(DATA_WIDTH - 1 downto 0)
        );
end RAM;

architecture behavioral of RAM is

        type RAM_TYPE is array (0 to 2**ADDRESS_WIDTH - 1) of std_logic_vector(DATA_WIDTH - 1 downto 0);
        signal ram : RAM_TYPE;

begin

        process(address, wr)
        begin
                dout <= ram(conv_integer(unsigned(address)));

                if (wr='1') then
                        ram(conv_integer(unsigned(address))) <= din;
                end if;
        end process;

end behavioral;
```

A.30 OutputBuffer

```
library ieee;
use ieee.std_logic_1164.all;

entity OutputBuffer is
        port(
        D : in std_logic_vector(3 downto 0);
        Load : in std_logic;
        ACK : in std_logic;
```

```
        RESET : in std_logic;
        CLK : in std_logic;
        OBfree : out std_logic;
        Dval : out std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end OutputBuffer;

architecture structural of OutputBuffer is

component BufferControl is
        port(
        Load : in std_logic;
        ACK : in std_logic;
   RESET : in std_logic;
   CLK : in std_logic;
   Wreg : out std_logic;
        OBfree : out std_logic;
        Dval : out std_logic
        );
end component;

component OutputRegister is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end component;

signal Wreg_Output : std_logic;

begin
U0 : BufferControl port map( Load => Load, ACK => ACK, RESET => RESET, CLK => CLK,
Wreg => Wreg_Output, OBfree => OBfree, Dval => Dval );

U1 : OutputRegister port map( CLK => Wreg_Output, RESET => RESET, D => D, EN => '1',
Q => Q);

end structural;
```

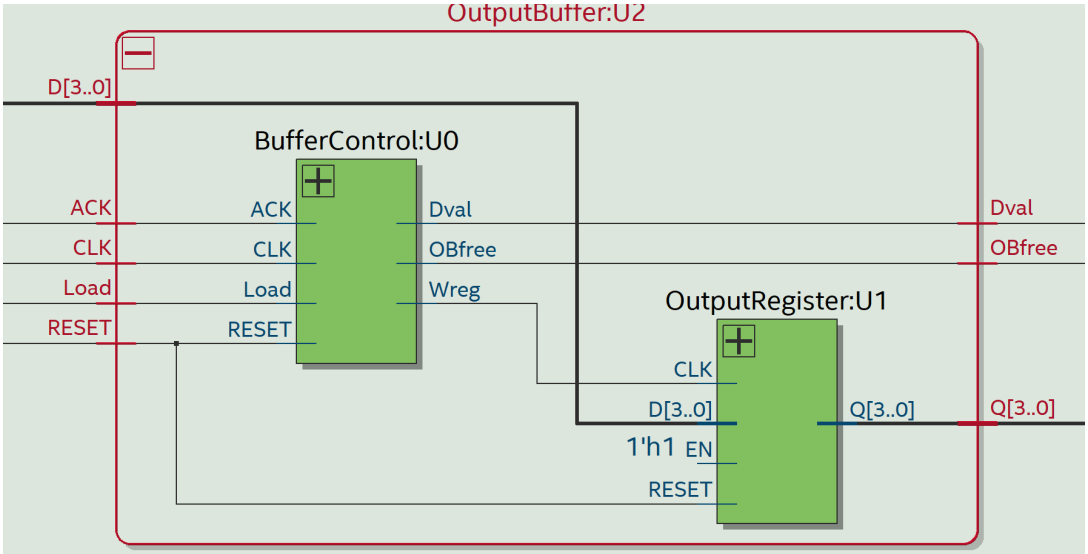


Figura 19: Netlist Viewer do *OutputBuffer*

A.31 BufferControl

```
library ieee;
use ieee.std_logic_1164.all;

entity BufferControl is
        port(
        Load : in std_logic;
        ACK : in std_logic;
   RESET : in std_logic;
   CLK : in std_logic;
   Wreg : out std_logic;
        OBfree : out std_logic;
        Dval : out std_logic
        );
end BufferControl;

architecture behavioral of BufferControl is

type STATE_TYPE is (INITIAL_STATE, LOAD_DATA, VALID_DATA, WAIT_ACK0);

signal CurrentState, NextState : STATE_TYPE;

begin

CurrentState <= INITIAL_STATE when RESET = '1' else NextState when rising_edge(CLK);

GenerateNextState:
process (CurrentState, Load , ACK)
        begin
                case CurrentState is
                        when INITIAL_STATE        => if (Load = '1') then
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
        NextState <= LOAD_DATA;
                                else
        NextState <= INITIAL_STATE;
                                end if;
```

```
                        when LOAD_DATA                  => if (Load = '0') then
                                                                        NextState <= VALID_DATA;
                                                                else
                                                                        NextState <= LOAD_DATA;
                                                                end if;

                        when VALID_DATA                 => if (ACK = '1') then
                                                                        NextState <= WAIT_ACKO;
                                                                else
                                                                        NextState <= VALID_DATA;
                                                                end if;

                        when WAIT_ACKO                  => if (ACK = '0') then
                                                                        NextState <= INITIAL_STATE;
                                                                else
                                                                        NextState <= WAIT_ACKO;
                                                                end if;
                end case;
        end process;

OBfree <= '1' when (CurrentState = INITIAL_STATE) else '0';
Wreg <= '1' when (CurrentState = LOAD_DATA) else '0';
Dval <= '1' when (CurrentState = VALID_DATA) else '0';

end behavioral;
```

A.32 OutputRegister

```
library ieee;
use ieee.std_logic_1164.all;

entity OutputRegister is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        D : in std_logic_vector(3 downto 0);
        EN : in std_logic;
        Q : out std_logic_vector(3 downto 0)
        );
end OutputRegister;

architecture structural of OutputRegister is

component FFD1 is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN : in std_logic;
        Q : out std_logic
        );
end component;

begin

U0: FFD1 port map(CLK => CLK, RESET => RESET, SET => '0', D => D(0), EN => EN, Q => Q(0));
```

```
U1: FFD1 port map(CLK => CLK, RESET => RESET, SET => '0', D => D(1), EN => EN, Q => Q(1));
U2: FFD1 port map(CLK => CLK, RESET => RESET, SET => '0', D => D(2), EN => EN, Q => Q(2));
U3: FFD1 port map(CLK => CLK, RESET => RESET, SET => '0', D => D(3), EN => EN, Q => Q(3));

end structural;
```

A.33 FFD1

```
library ieee;
use ieee.std_logic_1164.all;

entity FFD1 is
        port(
        CLK : in std_logic;
        RESET : in std_logic;
        SET : in std_logic;
        D : in std_logic;
        EN: in std_logic;
        Q : out std_logic
        );
end FFD1;

architecture structural of FFD1 is

begin


Q <= '0' when RESET = '1' else '1' when SET = '1' else D when rising_edge(CLK) and EN = '1';

end structural;
```

B Kotlin

B.1 HAL

Algoritmo 1: HAL - Hardware Abstract Layer

```
import isel.leic.UsbPort

object HAL {
    private var currentState = 0x00

    // Inicia o objeto
    fun init() = UsbPort.write(currentState)

    // Retorna 'true' se o bit definido pela mask esta com o valor logico '1' no UsbPort
    fun isBit(mask: Int): Boolean{
        return (UsbPort.read() and mask) != 0
    }

    // Retorna os valores dos bits representados por mask presentes no UsbPort
    fun readBits(mask: Int): Int{
        return UsbPort.read() and mask
    }

    // Escreve nos bits representados por mask os valores dos bits correspondentes em value
    fun writeBits(mask: Int, value: Int){
        clrBits(mask)
        setBits(value and mask)
    }

    // Coloca os bits representados por mask no valor logico '1'
    fun setBits(mask: Int){
        currentState = currentState or mask
        UsbPort.write(currentState)  // Atualiza o estado no hardware
    }

    // Coloca os bits representados por mask no valor logico '0'
    fun clrBits(mask: Int){
        currentState = currentState and mask.inv()
        UsbPort.write(currentState)  // Atualiza o estado no hardware
    }
}

fun main(){
    HAL.init() // Inicializa o HAL (estado inicial = 0)
    HAL.isBit(0x01) // Verifica se o bit 0 esta ativo
    HAL.readBits(0xFF) // Le o estado atual de todos os bits
    HAL.writeBits(0xF0 , 0xA0) // Escreve 1010_0000 nos bits 4 a 7 (com mascara 0xF0)
    HAL.setBits(0x0F) // Ativa os primeiros 4 bits (bits 0 a 3)
    HAL.clrBits(0x05) // Limpa os bits 0 e 2 (mascara 0000_0101)
    println("Bit 1 ativo? ${HAL.isBit(0x02)}") // Exibe o estado final do bit 1
}
```

B.2 KBD

Algoritmo 2: KBD

```
import isel.leic.utils.Time

// Ler teclas. Funcoes retornam    0   ..   9   ,    A   ..   D   ,    #    ,   *    ou NONE.
object KBD {
    const val NONE = 0
    const val DVAL_MASK = 0x10
    const val D_MASK = 0x0F
    const val ACK_MASK = 0x80
    const val TECLADO = "147*2580369#ABCD"

    // Inicia a classe
    fun init() = HAL.clrBits(ACK_MASK)

    // Retorna de imediato a tecla premida ou NONE se nao ha tecla premida.
```

```
    fun getKey(): Char{
        if (!HAL.isBit(DVAL_MASK)) return NONE.toChar()

        val index = HAL.readBits(D_MASK)
        val key = TECLADO[index]

        HAL.setBits(ACK_MASK)
        while(HAL.isBit(DVAL_MASK)); // Aguarda que a tecla seja solta
        HAL.clrBits(ACK_MASK)

        return key
    }

    // Retorna a tecla premida, caso ocorra antes do     timeout     (em milissegundos),
    // ou NONE caso contrario.
    fun waitKey(timeout: Long): Char{
        val endTime = Time.getTimeInMillis() + timeout
        do {
            val key = getKey()
            if (key != NONE.toChar()) return key
        } while (Time.getTimeInMillis()  < endTime )

        return NONE.toChar() // Retorna NONE se o tempo acabar
    }
}

fun main() {
    HAL.init()
    KBD.init()
    while(true) {
        val key = KBD.waitKey(5000) // Espera ate 5 segundos por uma tecla
        if (key != KBD.NONE.toChar()) { // So imprime se for uma tecla valida
            println("Tecla␣pressionada:␣$key")
        }
    }
}
```

C Atribuição de Pinos

```
set_global_assignment -name BOARD "MAX␣10␣DE10␣-␣Lite"
set_global_assignment -name DEVICE 10M50DAF484C6GES
set_global_assignment -name FAMILY "MAX␣10"

#CLOCK
set_location_assignment PIN_P11 -to CLK

#RESET
set_location_assignment PIN_F15 -to RESET

#KEYPAD
set_location_assignment PIN_W5   -to KeyPadL[0]
set_location_assignment PIN_AA14 -to KeyPadL[1]
set_location_assignment PIN_W12  -to KeyPadL[2]
set_location_assignment PIN_AB12 -to KeyPadL[3]
set_location_assignment PIN_AB11 -to KeyPadC[0]
set_location_assignment PIN_AB10 -to KeyPadC[1]
set_location_assignment PIN_AA9  -to KeyPadC[2]
set_location_assignment PIN_AA8  -to KeyPadC[3]

#Dval e D
set_location_assignment PIN_A8  -to D[0]
set_location_assignment PIN_A9  -to D[1]
set_location_assignment PIN_A10 -to D[2]
set_location_assignment PIN_B10 -to D[3]
set_location_assignment PIN_D13 -to Dval

#LCD
set_location_assignment PIN_W11    -to DLCD[4]
set_location_assignment PIN_AA10   -to DLCD[5]
set_location_assignment PIN_Y8     -to DLCD[6]
set_location_assignment PIN_Y7     -to DLCD[7]
set_location_assignment PIN_W8     -to RS
set_location_assignment PIN_V5     -to E

#Display
#============================================================
# HEX0
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX0[7]
set_location_assignment PIN_C14 -to HEX0[0]
set_location_assignment PIN_E15 -to HEX0[1]
set_location_assignment PIN_C15 -to HEX0[2]
set_location_assignment PIN_C16 -to HEX0[3]
set_location_assignment PIN_E16 -to HEX0[4]
```

```
set_location_assignment PIN_D17 -to HEX0[5]
set_location_assignment PIN_C17 -to HEX0[6]
set_location_assignment PIN_D15 -to HEX0[7]

#============================================================
# HEX1
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX1[7]
set_location_assignment PIN_C18 -to HEX1[0]
set_location_assignment PIN_D18 -to HEX1[1]
set_location_assignment PIN_E18 -to HEX1[2]
set_location_assignment PIN_B16 -to HEX1[3]
set_location_assignment PIN_A17 -to HEX1[4]
set_location_assignment PIN_A18 -to HEX1[5]
set_location_assignment PIN_B17 -to HEX1[6]
set_location_assignment PIN_A16 -to HEX1[7]

#============================================================
# HEX2
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX2[7]
set_location_assignment PIN_B20 -to HEX2[0]
set_location_assignment PIN_A20 -to HEX2[1]
set_location_assignment PIN_B19 -to HEX2[2]
set_location_assignment PIN_A21 -to HEX2[3]
set_location_assignment PIN_B21 -to HEX2[4]
set_location_assignment PIN_C22 -to HEX2[5]
set_location_assignment PIN_B22 -to HEX2[6]
set_location_assignment PIN_A19 -to HEX2[7]

#============================================================
# HEX3
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[6]
```

```
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX3[7]
set_location_assignment PIN_F21 -to HEX3[0]
set_location_assignment PIN_E22 -to HEX3[1]
set_location_assignment PIN_E21 -to HEX3[2]
set_location_assignment PIN_C19 -to HEX3[3]
set_location_assignment PIN_C20 -to HEX3[4]
set_location_assignment PIN_D19 -to HEX3[5]
set_location_assignment PIN_E17 -to HEX3[6]
set_location_assignment PIN_D22 -to HEX3[7]

#============================================================
# HEX4
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX4[7]
set_location_assignment PIN_F18 -to HEX4[0]
set_location_assignment PIN_E20 -to HEX4[1]
set_location_assignment PIN_E19 -to HEX4[2]
set_location_assignment PIN_J18 -to HEX4[3]
set_location_assignment PIN_H19 -to HEX4[4]
set_location_assignment PIN_F19 -to HEX4[5]
set_location_assignment PIN_F20 -to HEX4[6]
set_location_assignment PIN_F17 -to HEX4[7]

#============================================================
# HEX5
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[0]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[1]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[2]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[3]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[4]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[5]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[6]
set_instance_assignment -name IO_STANDARD "3.3-V␣LVTTL" -to HEX5[7]
set_location_assignment PIN_J20 -to HEX5[0]
set_location_assignment PIN_K20 -to HEX5[1]
set_location_assignment PIN_L18 -to HEX5[2]
set_location_assignment PIN_N18 -to HEX5[3]
set_location_assignment PIN_M20 -to HEX5[4]
set_location_assignment PIN_N19 -to HEX5[5]
set_location_assignment PIN_N20 -to HEX5[6]
set_location_assignment PIN_L19 -to HEX5[7]
```

	tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✔ ...		CLK	Location	PIN_P11	Yes			
2	✔ ...		R...T	Location	PIN_F15	Yes			
3	✔ ...		Ke...0]	Location	PIN_W5	Yes			
4	✔ ...		Ke...1]	Location	PI...14	Yes			
5	✔ ...		Ke...2]	Location	PI...12	Yes			
6	✔ ...		Ke...3]	Location	PI...12	Yes			
7	✔ ...		Ke...0]	Location	PI...11	Yes			
8	✔ ...		Ke...1]	Location	PI...10	Yes			
9	✔ ...		Ke...2]	Location	PI...A9	Yes			
10	✔ ...		Ke...3]	Location	PI...A8	Yes			
11	✔ ...		Dval	Location	PI...13	Yes			
12	✔ ...		D[0]	Location	PIN_A8	Yes			
13	✔ ...		D[1]	Location	PIN_A9	Yes			
14	✔ ...		D[2]	Location	PI...10	Yes			
15	✔ ...		D[3]	Location	PIN_B10	Yes			
16	✔ ...		DL...4]	Location	PI...11	Yes			
17	✔ ...		DL...5]	Location	PI...10	Yes			
18	✔ ...		DL...6]	Location	PIN_Y8	Yes			
19	✔ ...		DL...7]	Location	PIN_Y7	Yes			
20	✔ ...		RS	Location	PIN_W8	Yes			
21	✔ ...		E	Location	PIN_V5	Yes			
22	✔ ...		HE...0]	Location	PIN_C14	Yes			
23	✔ ...		HE...1]	Location	PIN_E15	Yes			
24	✔ ...		HE...2]	Location	PIN_C15	Yes			
25	✔ ...		HE...3]	Location	PIN_C16	Yes			

Figura 20: Assignment Editor - parte 1

26	✔ ...		HE...4]	Location	PIN_E16	Yes			
27	✔ ...		HE...5]	Location	PI...17	Yes			
28	✔ ...		HE...6]	Location	PIN_C17	Yes			
29	✔ ...		HE...7]	Location	PI...15	Yes			
30	✔ ...		HE...0]	Location	PIN_C18	Yes			
31	✔ ...		HE...1]	Location	PI...18	Yes			
32	✔ ...		HE...2]	Location	PIN_E18	Yes			
33	✔ ...		HE...3]	Location	PIN_B16	Yes			
34	✔ ...		HE...4]	Location	PI...17	Yes			
35	✔ ...		HE...5]	Location	PI...18	Yes			
36	✔ ...		HE...6]	Location	PIN_B17	Yes			
37	✔ ...		HE...7]	Location	PI...16	Yes			
38	✔ ...		HE...0]	Location	PIN_B20	Yes			
39	✔ ...		HE...1]	Location	PI...20	Yes			
40	✔ ...		HE...2]	Location	PIN_B19	Yes			
41	✔ ...		HE...3]	Location	PI...21	Yes			
42	✔ ...		HE...4]	Location	PIN_B21	Yes			
43	✔ ...		HE...5]	Location	PIN_C22	Yes			
44	✔ ...		HE...6]	Location	PIN_B22	Yes			
45	✔ ...		HE...7]	Location	PI...19	Yes			
46	✔ ...		HE...0]	Location	PIN_F21	Yes			
47	✔ ...		HE...1]	Location	PIN_E22	Yes			
48	✔ ...		HE...2]	Location	PIN_E21	Yes			
49	✔ ...		HE...3]	Location	PIN_C19	Yes			

Figura 21: Assignment Editor - parte 2

50	✔ ...		HE...4]	Location	PIN_C20	Yes			
51	✔ ...		HE...5]	Location	PI...19	Yes			
52	✔ ...		HE...6]	Location	PIN_E17	Yes			
53	✔ ...		HE...7]	Location	PI...22	Yes			
54	✔ ...		HE...0]	Location	PIN_F18	Yes			
55	✔ ...		HE...1]	Location	PIN_E20	Yes			
56	✔ ...		HE...2]	Location	PIN_E19	Yes			
57	✔ ...		HE...3]	Location	PIN_J18	Yes			
58	✔ ...		HE...4]	Location	PI...19	Yes			
59	✔ ...		HE...5]	Location	PIN_F19	Yes			
60	✔ ...		HE...6]	Location	PIN_F20	Yes			
61	✔ ...		HE...7]	Location	PIN_F17	Yes			
62	✔ ...		HE...0]	Location	PIN_J20	Yes			
63	✔ ...		HE...1]	Location	PIN_K20	Yes			
64	✔ ...		HE...2]	Location	PIN_L18	Yes			
65	✔ ...		HE...3]	Location	PI...18	Yes			
66	✔ ...		HE...4]	Location	PI...20	Yes			
67	✔ ...		HE...5]	Location	PI...19	Yes			
68	✔ ...		HE...6]	Location	PI...20	Yes			
69	✔ ...		HE...7]	Location	PIN_L19	Yes			
70		<<...>>	<<...>>	<<new>>					

Figura 22: Assignment Editor - parte 3